

NFC Smart Lock

Mobile Application Engineering Specification

Wire Protocol, Session Control, Identity Storage, and State Management (Phone)

Version 0.1 – Draft (Pre-Release)

August 4, 2026

Abstract

This document is the engineering specification for the Phone-side (Flutter) mobile application of the NFC Smart Lock system. It defines, as normative requirements, the wire protocol the application speaks, the session state machine and transport contract, identity and trusted-lock persistence, and the dependency-injection architecture binding these together – without describing how any component is implemented internally. Implementation detail (source file locations, class internals, and Dart code) is intentionally excluded from this document; where a companion implementation reference is available, it is referenced by subject rather than by path. Presentation (screens, widgets, visual design) is out of scope for this document by request; see the project’s UI design reference separately.

This document is the Phone-side counterpart to the *Firmware Engineering Specification* and shares its wire contract: the SLOCK-HS-v1 handshake (Part I there, Part I here) and the application command set (Part II there, Part III here). Where the two documents’ figures for a shared constant disagree, this document states the figure this application actually enforces and flags the discrepancy explicitly rather than silently picking a side; see §2.

Contents

Document Map and Reading Order	4
I Wire Protocol (SLOCK-HS-v1)	5
1 Scope	5
2 Protocol Constants	5
3 APDU Framing	6
3.1 Command APDU: Phone → Lock	6
3.2 Response APDU: Lock → Phone	6
3.3 Instruction Set	7
3.4 Status Words (SW1/SW2)	7
4 The Three-Message Mutual Authentication Handshake	8
4.1 Message Flow	8
4.2 Message Definitions	8

4.3	The Transcript	8
4.4	Verification Rules (Phone Side)	8
4.5	Handshake API Contract	9
5	Cryptographic Primitives	9
6	Session Keys and Secure Communication	10
6.1	Key Derivation	10
6.2	AES-256-GCM Message Format	10
6.3	Plaintext Capacity	10
6.4	Replay Protection	10
II	Session Control and Transport	11
1	The Transport Abstraction	11
1.1	Interface Contract	11
1.2	Transport Events and Errors	11
1.3	Platform Binding (Android)	11
1.4	Test Double	12
2	Session Controller	12
2.1	Construction Contract	12
2.2	State Machine	13
2.3	Normal Session Execution	13
3	Secure Command Exchange	14
4	Session Teardown	14
4.1	Abort	14
4.2	State Release	15
III	Provisioning and Identity	16
1	Provisioning: The Deferred-Verification Flow	16
1.1	Why Verification Is Deferred, Not Skipped	16
1.2	Flow	16
1.3	Provision Session Restrictions	17
2	Application Commands	17
2.1	Command Table	17
2.2	Application Status Bytes	17
2.3	Lock State Bytes	18
3	Identity and Trusted-Lock Storage	18
3.1	Phone Identity Store	18
3.2	Trusted-Lock Store	18

IV	Application State Architecture	19
1	Overview and Design Assumptions	19
2	Dependency Injection Graph	19
2.1	Provider Registry	20
2.2	Dependency Graph	20
3	Reactive Trusted-Lock State	21
3.1	State Shape and Transitions	21
3.2	Required Operations	21
4	Testing Contract	21
V	Timing, Errors, and Guardrails	22
1	Timing and Session Guardrails	22
2	Error Model	22
2.1	Error Taxonomy	22
2.2	Status-Word to Error Mapping	23
2.3	Transport Error to Application Error Mapping	23
VI	Conformance and Security Properties	24
1	Mapping to the Firmware Specification	24
2	Flagged Cross-Document Reconciliations	24
2.1	Resolved in Practice: Plaintext Capacity (Firmware OI-1)	24
2.2	Not Yet Resolved: C-APDU Framing (Le Byte)	25
3	Security Properties	25
VII	Document Set History	26
1	Provenance and Scope of This Revision	26
2	Open Items Tracked by This Revision	26
3	Revision History	26

Document Map and Reading Order

Part	Covers
I. Wire Protocol	APDU framing, the SLOCK-HS-v1 handshake, cryptographic primitives, secure messaging
II. Session Control & Transport	The transport abstraction, the session state machine, normal (non-provisioning) session execution
III. Provisioning & Identity	The deferred-verification provisioning flow, application command set, and persistent identity/trust storage
IV. Application State Architecture	The dependency-injection graph and reactive state exposed to (but not including) the presentation layer
V. Timing, Errors, and Guardrails	Session guardrails and the complete error model
VI. Conformance and Security Properties	Mapping to the firmware specification, security properties, and flagged cross-document discrepancies
VII. Document Set History	Provenance of this document

Numbering note: consistent with the Firmware Engineering Specification, this project has not reached a stable v1.0 release. This document's version is independent of, and not comparable to, the firmware document's version.

Scope note: this document deliberately omits the presentation layer (screens, widgets, navigation, visual design). It covers everything below that layer: protocol, session orchestration, persistence, and the state/dependency-injection architecture that a presentation layer would consume.

Part I

Wire Protocol (SLOCK-HS-v1)

1 Scope

This part specifies the Phone-side implementation of SLOCK-HS-v1: a bespoke, zero-trust, mutual-authentication, end-to-end-encrypted protocol running over ISO/IEC 7816-4 short APDUs on an ISO/IEC 14443-4 (ISO-DEP) contactless link. It is normative for the mobile application and is the counterpart to the Firmware Engineering Specification's Part I (Session Module) and Part IV (Transport); the two documents specify opposite ends of the same wire contract.

2 Protocol Constants

Constant	Value	Purpose
CLA	0x80	Proprietary class byte
P1 / P2	0x00	Reserved; a shared convention with the firmware, not independently validated by this application's transport
Handshake-init INS	0x10	Carries M1
Handshake-finish INS	0x11	Carries M3
Secure-payload INS	0x20	Carries an encrypted application command
Session-abort INS	0x30	Best-effort session teardown
Transcript version byte	0x01	Handshake transcript version
Max command data	255 bytes	Short-APDU C-APDU data limit
Max response data	256 bytes	Short-APDU R-APDU data limit
Transport Lc ceiling	227 bytes	Enforced ceiling for the complete encrypted package (nonce ciphertext tag)
GCM nonce length	12 bytes	
GCM tag length	16 bytes	
Max secure plaintext	199 bytes	Transport Lc ceiling minus 28 bytes of AEAD overhead; see §5.3
X25519 key length	32 bytes	
Ed25519 public key length	32 bytes	
Ed25519 signature length	64 bytes	
Challenge length	32 bytes	
M1 / M2 / M3 length	64 / 128 / 64 bytes	
Transcript prefix	"SLOCK-HS-v1"	Domain separator
HKDF info (Phone→Lock)	"phone->esp"	

Constant	Value	Purpose
HKDF (Lock→Phone)	info	"esp→phone"
Derived key length	32 bytes	AES-256 key size

3 APDU Framing

Communication uses Command APDUs (C-APDU) and Response APDUs (R-APDU) under ISO/IEC 7816-4 Short APDU **Case 1 / Case 3** conventions – **this application never appends a trailing Le byte**; see §2 for how this relates to the firmware specification’s stated C-APDU bound.

3.1 Command APDU: Phone → Lock

Field	Size	Value
CLA	1 byte	0x80
INS	1 byte	0x10/0x11/0x20/0x30
P1	1 byte	0x00
P2	1 byte	0x00
Lc	0–1 byte	Length of data payload; omitted entirely when the payload is empty
Data	<i>Lc</i> bytes	Handshake parameters or AES-GCM secure payload

Case 1 (no data, e.g. CMD_SESSION_ABORT): [CLA INS P1 P2] – 4 bytes total, Lc omitted.
Case 3 (with data): [CLA INS P1 P2 Lc Data] – 5 + *Lc* bytes total, maximum 260 bytes at *Lc* = 255.

3.2 Response APDU: Lock → Phone

Field	Size	Value
Data	0–256 bytes	Handshake response or encrypted application response
SW1	1 byte	0x90 = normal processing
SW2	1 byte	Status sub-code

The application SHALL treat the final two bytes of any received R-APDU as SW1/SW2 unconditionally; every byte before them is response data.

3.3 Instruction Set

Instruction	INS	Carries	Valid State
CMD_HANDSHAKE_INIT	0x10	M1, 64 bytes	activated
CMD_HANDSHAKE_FINISH	0x11	M3, 64 bytes	handshake
CMD_SECURE_PAYLOAD	0x20	Encrypted application command	secureSession
CMD_SESSION_ABORT	0x30	none	any active state

Design rationale: a single secure envelope. All application commands are funnelled through one INS = 0x20 rather than assigning each command its own instruction code. This keeps the transport-layer state machine blind to application business logic – it enforces only that 0x20 is legal after a secure session is established. The application opcode lives inside the decrypted plaintext, a distinct namespace from INS; see Part III §2.

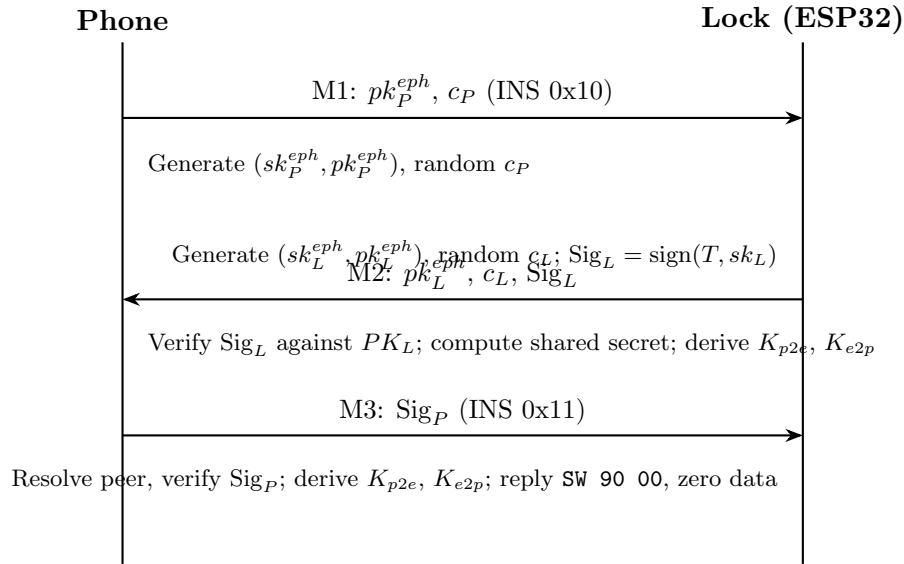
3.4 Status Words (SW1/SW2)

SW1	SW2	Mnemonic	Meaning
0x90	0x00	ok	Success
0x69	0x82	authFailed	Signature or AEAD tag verification failed
0x69	0x85	wrongState	INS not valid in the current transport state
0x6A	0x80	invalidData	Lc mismatch or payload too large
0x6A	0x81	notSupported	Unknown INS, or CLA $\neq$ 0x80

Any status-word pair not in this table SHALL be treated as an unknown status, distinct from a recognized failure (§2).

4 The Three-Message Mutual Authentication Handshake

4.1 Message Flow



4.2 Message Definitions

Msg	Sender	Contents
M1	Phone	pk_P^{eph} (X25519, 32 B) $\parallel$ c_P (32 B). Carries no identity hint.
M2	Lock	pk_L^{eph} (32 B) $\parallel$ c_L (32 B) $\parallel$ Sig_L (Ed25519, 64 B)
M3	Phone	Sig_P (Ed25519, 64 B)

4.3 The Transcript

To prevent replay, splicing, and downgrade attacks, every signature is generated over a domain-separated transcript:

$T = \text{"SLOCK-HS-v1"} \parallel 0x01 \parallel pk_eph_P \parallel pk_eph_L \parallel c_P \parallel c_L$ (140 bytes)

Both platforms SHALL compute an identical transcript, so no signature is valid outside the session it was created for.

4.4 Verification Rules (Phone Side)

1. Upon receiving M2 in a **normal** (non-provisioning) session, the Phone MUST verify Sig_L against the lock's resolved long-term Ed25519 public key (Part III §3) before proceeding to M3. On failure it MUST abort without deriving any session key material. (During provisioning, this verification is deferred rather than skipped; see Part III §1.)
2. The Phone MUST reject a handshake if pk_P^{eph} or pk_L^{eph} is the all-zero point or otherwise fails X25519 validity checks.
3. c_P MUST be generated with a CSPRNG and MUST NOT be reused across sessions.

4. The M3 acknowledgement MUST be **SW 90 00** with **zero** data bytes. Any data present in the M3 acknowledgement is a protocol violation and MUST be treated as an unexpected response, not silently ignored.

4.5 Handshake API Contract

Operation	Contract
Build M1	Generates a fresh ephemeral X25519 keypair and challenge c_P ; returns the 64-byte M1 payload plus opaque handshake context to be threaded through the remainder of the exchange.
Parse and verify M2	Parses M2, verifies Sig_L against a supplied lock public key, computes the shared secret. Used for normal sessions.
Cache M2 for provisioning	Parses M2 without verifying Sig_L , retaining it for deferred verification. Used only during provisioning (Part III §1).
Build M3	Produces the 64-byte M3 payload (Sig_P) and derives both directional session keys from the transcript, the shared secret, and both challenges.
Verify deferred M2	Verifies a previously cached, unverified Sig_L against a lock public key obtained later (from a CMD_PROVISION response). Returns whether the cached signature is valid for that specific key.

The handshake context, the parsed-M2 result, and the deferred-M2 cache are opaque data owned by the Session Controller (Part II) across the $M1 \rightarrow M2 \rightarrow M3$ exchange; none of them are valid outside the session that produced them.

5 Cryptographic Primitives

Primitive	Use	Notes
Ed25519 (RFC 8032)	Long-term identity signatures over the handshake transcript	32-byte seed, 32-byte public key
X25519 (RFC 7748)	Ephemeral key agreement	All-zero output MUST be rejected
HKDF-SHA256 (RFC 5869)	Session-key derivation	Extract+Expand; challenges bound into the salt
AES-256-GCM	Authenticated encryption of application payloads	12-byte nonce, 16-byte tag, empty AAD
CSPRNG	Nonces, ephemeral keys, challenges	

All primitives SHALL be pure and side-effect-free with respect to application state. No custom cryptographic primitives are implemented.

6 Session Keys and Secure Communication

6.1 Key Derivation

```
PRK    = HKDF-Extract(salt = c_P || c_L, ikm = X25519 shared secret)
K_p2e  = HKDF-Expand(PRK, info = "phone->esp", L = 32)
K_e2p  = HKDF-Expand(PRK, info = "esp->phone", L = 32)
```

Design rationale: directional keys. Two separate keys are derived, rather than one master key, specifically to prevent reflection attacks: if an attacker records an encrypted phone-originated message and replays it back to the phone, the phone attempts to decrypt it with K_{e2p} (the lock-originated key), and AES-GCM tag verification fails immediately.

6.2 AES-256-GCM Message Format

```
Payload = Nonce(12) || Ciphertext || Tag(16)
```

Nonce: 12 bytes, CSPRNG-generated, unique per message per key. AAD: empty (this holds on both ends of the wire contract). Outbound messages SHALL be sealed with K_{p2e} ; inbound messages SHALL be opened with K_{e2p} , and a tag mismatch MUST be treated as a fatal, non-retryable session failure (§2).

6.3 Plaintext Capacity

The maximum secure plaintext this application will encrypt in one message is **199 bytes**, computed as the 227-byte transport Lc ceiling minus 28 bytes of AEAD overhead (12-byte nonce + 16-byte tag). This matches the firmware’s as-built enforcement (a `CMD_SECURE_PAYLOAD` C-APDU with Lc > 227 is rejected with SW 6A 80). **This is a deliberate adoption of the conservative, arithmetically-consistent figure** in preference to an earlier, differently-derived 227-byte plaintext figure that appeared in an early draft of the firmware specification; see §2 for the full discussion. A plaintext exceeding 199 bytes MUST be rejected locally before any wire transmission is attempted.

6.4 Replay Protection

Each session derives a fresh key and is short-lived and single-purpose; sequence-number-based replay protection within a session is therefore OPTIONAL at the protocol level. Whether a given plaintext command is safe to replay is an application-layer policy decision. The handshake itself is replay-protected by fresh challenges bound into the signed transcript.

Part II

Session Control and Transport

1 The Transport Abstraction

All NFC communication SHALL be reached through a single abstract transport contract, decoupling the rest of the application from any OS-specific NFC implementation and permitting a scriptable substitute for testing.

1.1 Interface Contract

Operation	Contract
Event stream	Emits a <i>tag discovered</i> event when an ISO-DEP tag enters the field and is ready for exchange, and a <i>tag lost</i> event when the connected tag leaves the field.
Start discovery	Begins polling for ISO-DEP tags. MUST fail with an NFC-unavailable error if the hardware is off or missing, rather than hanging or silently no-op-ing.
Stop discovery	Ends polling and, where the platform requires it, releases any active reader-mode configuration.
Transceive (command bytes)	Exchanges one raw C-APDU for one raw R-APDU. Precondition: a tag is currently connected; violating this precondition MUST produce a not-connected error, not an exception.
Connection query	Reports whether a tag is currently connected.
Disconnect	Releases the current tag connection, if any.

1.2 Transport Events and Errors

Event	Meaning
Tag discovered	An ISO-DEP tag entered the field and is ready for transceive
Tag lost	The connected tag was lost

Transport Error	Meaning
Not connected	No tag connected when a transceive was attempted
Tag lost	Tag lost during a transceive
Timeout	Transceive did not complete in time
Transceive failed	General transceive failure, carrying a diagnostic message
NFC unavailable	Hardware off or missing

1.3 Platform Binding (Android)

Requirement: the platform transport implementation MUST NOT perform an AID SELECT step – M1 (CMD_HANDSHAKE_INIT) is the first application-level APDU exchanged after ISO-DEP activation.

The connected-tag handle SHALL be treated as valid from tag discovery until either the tag is lost or discovery is explicitly stopped; the transport contract does not assume any additional explicit connect/close step beyond that. Transceive failures SHALL be classified into the transport error taxonomy above (tag-lost patterns take priority over a generic transceive-failed classification, followed by timeout-specific patterns, with anything else falling back to transceive-failed) so that callers above the transport never need platform-specific error handling.

1.4 Test Double

A scriptable transport substitute SHALL be available, supporting: driving connection events directly; enqueueing scripted responses (optionally validating the incoming command against an expected value); enqueueing a dynamically computed response derived from the incoming command (needed to produce a valid M2 in response to an arbitrary M1 during testing); enqueueing a specific transport error; and verifying that every enqueued interaction was consumed by the end of a test.

2 Session Controller

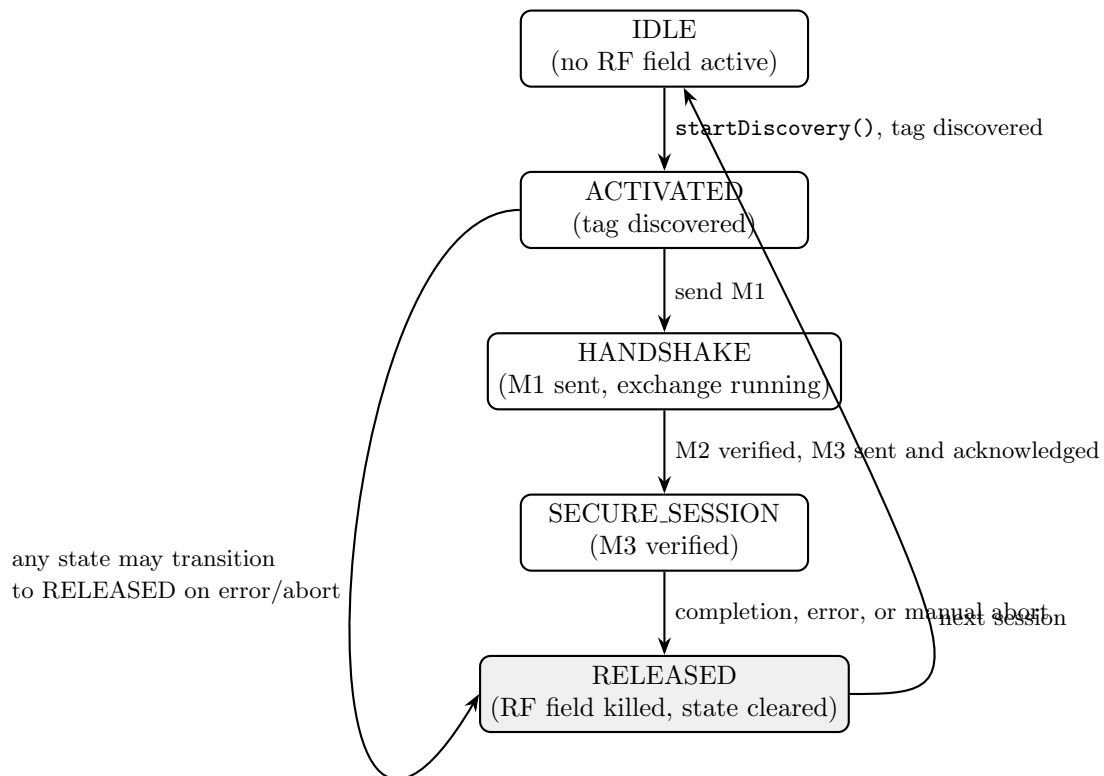
The Session Controller is the bridge between raw transport bytes and the cryptographic state machine. It owns the transport state machine (§2.2), the secure channel created after M3, and the handshake context created at M1.

2.1 Construction Contract

The Session Controller SHALL be constructed with exactly three dependencies:

- the transport (§1);
- a **lock key resolver**: given a lock identifier, returns the lock’s Ed25519 public key if trusted, or an explicit “unknown” result otherwise (Part III §3.2);
- a **phone identity resolver**: returns the phone’s long-term Ed25519 identity, generating and persisting one on first use if none exists (Part III §3.1).

2.2 State Machine



State	Meaning
idle	Initial state; no RF field active
activated	Discovery started; an ISO-DEP tag has entered the field
handshake	M1 sent; the M1/M2/M3 exchange is running
secureSession	M3 verified by the lock; secure channel initialized with K_{p2e}/K_{e2p}
released	Session ended (success, error, or manual abort); RF field killed

2.3 Normal Session Execution

Requirement: starting a session against a known lock identifier SHALL follow this sequence:

1. Reject with an invalid-state error if a session is already active.
2. Resolve the lock's Ed25519 public key. An unresolved (untrusted) lock MUST fail here, before any RF activity begins, not partway through the handshake.

3. Resolve the phone's long-term identity.
4. Run the handshake:
 - (a) Start discovery, then wait for a tag with a bounded timeout (§2.3 continues in Part V §1); transition to **activated**.
 - (b) Build and send M1.
 - (c) Parse the M2 R-APDU and **immediately** verify Sig_L against the resolved lock public key (Part I §4.5 rule 1). On failure: authentication-failed error, session released, RF field killed.
 - (d) Transition to **handshake**.
 - (e) Build and send M3. A non-success status maps to a handshake-specific error (§2); the acknowledgement **MUST** carry zero data bytes (Part I §4.5 rule 4) or the response is treated as unexpected.
 - (f) Construct the secure channel from the two derived directional keys; transition to **secureSession**.
5. The caller may now issue secure commands (§3) until it calls abort (§4).

3 Secure Command Exchange

Requirement: sending a secure application command **SHALL** follow this sequence:

1. Reject with an invalid-state error unless the session is currently in **secureSession**.
2. Encrypt the plaintext with K_{p2e} ; reject locally (before any wire transmission) if the plaintext exceeds 199 bytes (Part I §6.3).
3. Send the encrypted payload as an **INS 0x20 C-APDU**.
4. Parse the R-APDU. A non-success status maps to a secure-payload-specific error (§2); an AES-GCM tag mismatch is fatal to the session, not retryable with the same keys.
5. **An empty R-APDU data field is a valid, well-formed response and MUST be decrypted/returned as an empty plaintext, not treated as a transport-level error.** An empty successful application response corresponds to the firmware's documented behavior when its own application-response handoff times out internally (Firmware Engineering Specification, Part III §4.3); the application layer above this contract, not this layer, is responsible for treating an empty response as an application-level ambiguity.
6. Decrypt the response data with K_{e2p} . A tag mismatch **MUST** be treated identically to step 4's fatal case.

4 Session Teardown

4.1 Abort

Requirement: aborting a session **SHALL**:

1. If the session is in **activated**, **handshake**, or **secureSession**, send **CMD_SESSION_ABORT** best-effort – a transport failure at this step **MUST** be ignored, since the session is being torn down regardless of whether the lock receives this notice.

2. Release all session state (§5.2) and transition to **released**.
3. Stop discovery, ending the RF field.

4.2 State Release

Requirement: releasing session state SHALL transition to **released** and clear the secure channel and any deferred-M2 cache (Part III §1) unconditionally. This routine **MUST** be invoked on *every* error path, not only on an explicit user-initiated abort, so that no cryptographic material from a failed or aborted attempt can survive into the next session – this mirrors the firmware’s own secure-erase requirement (Firmware Engineering Specification, Part I §8).

Part III

Provisioning and Identity

1 Provisioning: The Deferred-Verification Flow

Provisioning associates a lock whose public key the phone does not yet know with the phone's own identity. This creates a chicken-and-egg problem: the phone must communicate securely with the lock before it can verify the lock is who it claims to be.

1.1 Why Verification Is Deferred, Not Skipped

Requirement: this application MUST NOT contain any code path in which handshake signature verification is unconditionally disabled. Verification is instead *deferred*: during a provisioning attempt, M2 is parsed and its signature is cached without being checked at that moment, and is checked moments later, once the lock's public key becomes known via `CMD_PROVISION`'s response. If verification were enforced synchronously at M2 during provisioning, provisioning would be structurally impossible, since the very key needed to check the signature is what provisioning exists to obtain.

1.2 Flow

1. **Physical intent (out of this document's scope):** the user triggers the lock's own provisioning window (a physical action on the lock, per the Firmware Engineering Specification Part II §7).
2. **Secret acquisition:** the application acquires a 32-byte Provision Secret from a QR code (or equivalent out-of-band channel). **Design rationale:** the acquired data contains *only* the secret, never the lock's public key, keeping the out-of-band payload minimal and relying entirely on the NFC channel to exchange actual identities.
3. **Session initiation:** the application starts discovery and waits for a tag, exactly as in a normal session (Part II §2.3).
4. **The handshake, with deferral:** M1 and M2 are exchanged identically to a normal session, except that M2's signature is cached (Part I §4.6, "cache M2 for provisioning") rather than verified. M3 proceeds and establishes what is, cryptographically, a fully functional but **not yet identity-verified** secure channel (§1.3).
5. **Authorization request:** the application encrypts and sends `CMD_PROVISION`, carrying the 32-byte secret and the phone's own Ed25519 public key.
6. **Lock-side validation (out of this document's scope):** the lock checks the secret, registers the phone's public key, and replies with its own public key (Firmware Engineering Specification, Part II §7.2).
7. **Deferred verification, completed:** the application takes the lock's public key from the `CMD_PROVISION` response and verifies the previously cached M2 signature against it (Part I §4.6, "verify deferred M2"). **This step MUST succeed before the lock is treated as trusted;** a `CMD_PROVISION` success response alone is not sufficient grounds to trust the lock's identity.

8. **Persistence:** only once the deferred signature check passes, the lock’s public key SHALL be committed to trusted-lock storage (§3.2), and any reactive trusted-locks state SHALL be refreshed so it is immediately visible above this layer.

1.3 Provision Session Restrictions

A provisioning-window session is **not** a general-purpose authenticated session, even though it uses an identical secure channel. Per the firmware’s own restriction (Firmware Engineering Specification, Part II §7.3), the lock will accept only `CMD_PROVISION` during its own provisioning window and will terminate the session if anything else is sent. This application **MUST NOT** send any command other than `CMD_PROVISION` while a provisioning attempt is in progress.

2 Application Commands

Once the secure channel is open, a one-byte opcode plus arguments is sent inside the encrypted plaintext (Part I §6). This opcode namespace is distinct from the transport INS byte introduced in Part I §2.3: the opcode lives inside decrypted plaintext and is never visible on the wire.

2.1 Command Table

Command	Opcode	Request Plaintext	Success Response
<code>CMD_PROVISION</code>	0x01	[0x01] Secret (32 B) Phone PK (32 B) – 65 B	[0x00] Lock PK (32 B) – 33 B
<code>CMD_UNLOCK</code>	0x02	[0x02] – 1 B	[0x00]
<code>CMD_LOCK</code>	0x03	[0x03] – 1 B	[0x00]
<code>CMD_GET_STATUS</code>	0x04	[0x04] – 1 B	[0x00] Battery % (1 B) Lock State (1 B) Last Error (1 B)
<code>CMD_REVOKE_KEY</code>	0x05	[0x05] Target Phone PK (32 B) – 33 B	[0x00]

2.2 Application Status Bytes

Value	Meaning
0x00	<code>APP_STATUS_OK</code> – command executed successfully
0x01	<code>APP_STATUS_INVALID_SECRET</code> – <code>CMD_PROVISION</code> only; the supplied secret did not match
0x02	<code>APP_STATUS_UNKNOWN_CMD</code> – opcode not present in §2.1
0x03	<code>APP_STATUS_ACTUATOR_FAULT</code> – the actuator reported an unresolved/unknown state

There is no 0x04 application status in the current protocol.

2.3 Lock State Bytes

Value	Meaning
0x00	Locked (bolt position confirmed)
0x01	Unlocked
0x02	Unknown (intermediate or unresolved position)

Any byte other than 0x00/0x01 MUST be interpreted as Unknown, not as an error.

3 Identity and Trusted-Lock Storage

3.1 Phone Identity Store

Requirement: the application SHALL persist exactly one long-term Ed25519 keypair for the phone itself (a 32-byte seed and a 32-byte public key), in platform-backed secure storage. The store SHALL support: retrieving the identity, generating and persisting one automatically on first use if none exists; checking whether an identity already exists; and deleting the identity (a factory-reset operation).

3.2 Trusted-Lock Store

Requirement: the application SHALL persist a mapping from lock identifier to the lock's Ed25519 public key, in platform-backed secure storage. The store SHALL support: resolving a lock identifier to a public key, or an explicit not-trusted result if absent; recording a new trusted key (written only after the deferred-verification check in §1.2 succeeds); removing a trusted key (used by `CMD_REVOKE_KEY`'s corresponding local-storage cleanup, §2.1); and listing all currently trusted lock identifiers.

Requirement: a lock's public key MUST NOT be written to this store at any point before its deferred M2 signature has been successfully verified (§1.2, step 7). A `CMD_PROVISION` success status alone MUST NOT trigger persistence.

Part IV

Application State Architecture

1 Overview and Design Assumptions

This part specifies how domain components (the Session Controller, identity and trusted-lock storage, and the higher-level domain facades built on them) are constructed, wired together, and exposed as reactive state – independent of, and without reference to, any presentation-layer consumer of that state.

- **Dependency injection:** domain components SHALL be constructed through a declarative provider graph rather than manual constructor plumbing, so that domain logic can be instantiated and exercised in isolation without a running presentation layer.
- **Global vs. local state:** this part governs only state that has meaning beyond a single screen or interaction (identity, trusted locks, domain controllers). Presentation-local, ephemeral state (in-progress flags, transient error messages, form input) is explicitly out of scope for this document.

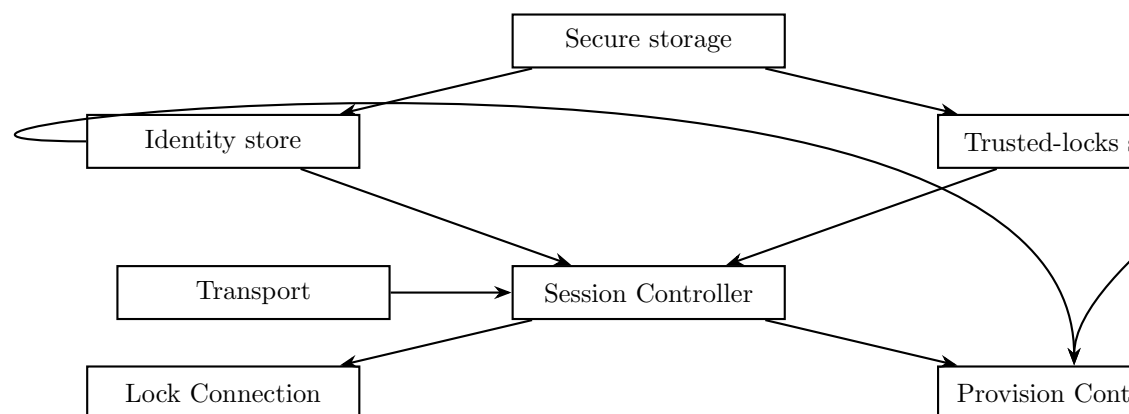
2 Dependency Injection Graph

Requirement: the application SHALL expose exactly one provider-graph container, established once at application start, serving as the single source of truth for every singleton and domain controller described below. All consumers of domain state or domain controllers SHALL resolve them from this graph rather than constructing their own instances.

2.1 Provider Registry

Provider	Category	Purpose
Logger	Core singleton	Application-wide diagnostic logging
NFC manager	Core singleton	Platform NFC manager handle
Transport	Core singleton	Bound to the platform ISO-DEP transport (Part II §1)
Secure storage	Core singleton	Hardware-backed keystore persistence substrate
Identity store	Core singleton	Phone's long-term Ed25519 identity (Part III §3.1)
Trusted-locks store	Core singleton	Trusted lock public keys (Part III §3.2)
Session Controller	Domain controller	Orchestrates the handshake and APDU flow (Part II)
Lock Connection	Domain controller	Typed high-level operations: unlock, lock, get status, revoke key
Provision Controller	Domain controller	Orchestrates the deferred-verification provisioning flow (§1)

2.2 Dependency Graph



Requirement: every domain controller's dependencies SHALL be resolved exclusively from this graph. No domain controller may construct a dependency (a store, the transport, or another controller) directly; doing so would defeat the single-source-of-truth guarantee this graph exists to provide, and would silently fork state between two independently constructed instances of what

should be one singleton.

3 Reactive Trusted-Lock State

Data that must be observed reactively above this layer – specifically, the current list of trusted locks – SHALL be exposed through a notifier-style reactive state container, not through a one-shot query.

3.1 State Shape and Transitions

State	Meaning
Loading	Initial state, and the state at the start of every refresh
Data (locks)	The current trusted lock identifiers, freshly re-read from the trusted-lock store
Error	A storage read or parse failure occurred

3.2 Required Operations

Operation	Contract
Refresh	Re-enters the Loading state, re-reads the trusted-lock store, and transitions to Data or Error based on the outcome. MUST be invoked automatically once the trusted-locks store is (re)constructed, and MUST be invoked again whenever the underlying store changes (§1.2 step 8; revoke, below).
Revoke	Removes a trusted key from the store, then performs Refresh so observers see the updated list without a separate, manually triggered reload.

Requirement: the reactive trusted-lock state SHALL be the *only* sanctioned way for anything above this layer to observe the current trusted-lock list. Nothing above this layer may read the trusted-lock store directly and bypass this reactive container, since doing so would produce state that silently drifts from what this container reports.

4 Testing Contract

Requirement: every domain controller specified in this part SHALL be constructible and independently testable without any presentation-layer dependency, given: the scriptable transport substitute (Part II §1.5); a substitute secure-storage backend satisfying the same contract as §3's stores. The complete provisioning-to-unlock cycle SHALL be exercisable end-to-end against a simulated lock peer that implements the wire protocol (Part I), independent of platform NFC hardware.

Part V

Timing, Errors, and Guardrails

1 Timing and Session Guardrails

Guardrail	Value	Enforced By
Tag-wait timeout	30 s	Session Controller, while awaiting tag discovery after <code>startDiscovery()</code>
Secure plaintext ceiling	199 bytes	Secure channel encryption path (Part I §6.3)
M3 acknowledgement data	exactly 0 bytes	Session Controller, handshake execution (Part II §2.3)
Command-response GCM tag	MUST verify	Secure channel decryption path
Session state transitions	Strictly ordered	Transport state machine and guard checks at every entry point (Part II §2.2–§3)

Requirement: the RF field SHALL be active only between the start of discovery and the point discovery is stopped (whether via abort or explicit cancellation). A user-triggered session that receives no tag within the 30-second tag-wait timeout MUST have its RF field dropped automatically, to prevent indefinite polling if the user walks away before tapping.

Requirement: the firmware enforces its own, independent handshake timeout (T_{HS} , recommended 2 s per the Firmware Engineering Specification Part IV §8.1), frame-integrity checks, and secure-erase routine. This application MUST tolerate a lock that tears a session down abruptly from the firmware side at any point – such an event will surface here as a tag-lost or unexpected-status condition, not as a distinguishable “the lock aborted deliberately” signal.

2 Error Model

All expected failure modes SHALL be modeled as a closed set of typed error values returned at layer boundaries, not as thrown exceptions; exceptions are reserved for programmer errors only.

2.1 Error Taxonomy

Error	Trigger
Tag lost	Tag lost during NFC communication
Timeout	An NFC transceive timed out, or the 30-second tag-wait elapsed
Unexpected status	The lock returned a status-word pair not in Part I §2.4’s table
Authentication failed	The lock’s M2 signature verification failed (normal session only)
Handshake rejected	The lock rejected M3 (SW 69 82) – a hard handshake failure, not retryable

Error	Trigger
Decryption failed	AES-GCM tag mismatch on a response – the session is dead; do not retry with the same keys
Invalid state	The requested operation is not valid in the current transport state
Untrusted lock	The requested lock’s public key is not in the trusted store
NFC unavailable	NFC hardware not available or disabled
Payload too large	Plaintext exceeds the 199-byte limit
Unexpected (diagnostic)	Any other unexpected condition, carrying a diagnostic message

2.2 Status-Word to Error Mapping

Requirement: the mapping from a received status word to an error value is context-dependent on the current handshake phase, and MUST be applied accordingly:

Phase	Mapping
M1/M2 (handshake init)	69 82 → authentication failed; 69 85 → invalid state; 6A 80/6A 81 → unexpected (diagnostic); anything else → unexpected status
M3 (handshake finish)	69 82 → handshake rejected; all other non-success statuses map as in the row above
Secure payload	69 82 → decryption failed (an AES-GCM failure; the session is dead)

2.3 Transport Error to Application Error Mapping

Transport Error	Maps To
Tag lost	Tag lost
Timeout	Timeout
Not connected	Invalid state
NFC unavailable	NFC unavailable
Transceive failed	Unexpected (diagnostic, carrying the underlying message)

Part VI

Conformance and Security Properties

1 Mapping to the Firmware Specification

The firmware is the ISO-DEP *target* (Lock) of this protocol; this application is the *initiator* (Phone).

This Application	Firmware Counterpart
Transport (Part II §1)	Firmware Part V (Hardware/Link Layer) + Part IV (Transport)
Session Controller / Handshake (Part I §4, Part II §2–§3)	Firmware Part I (Session Module: handshake, peer resolution)
Secure channel / crypto primitives (Part I §5)	Firmware Part I §6–§7 (key derivation, secure messaging)
Application commands / status bytes (Part III §2)	Firmware Part II §4–§5 (command set and status bytes)
Lock Connection / Provision Controller (Part III §1, Part IV)	Firmware Part II (dispatch, authorization, provisioning)

Wire contracts shared verbatim between the two documents: APDU framing (CLA 0x80, INS 0x10/0x11/0x20/ 0x30, P1/P2 0x00); handshake message sizes (M1 64 B, M2 128 B, M3 64 B) and transcript construction; HKDF-SHA256 key derivation with the salt and `info` strings of Part I §5.1; the AES-256-GCM payload shape of Part I §5.2; and the command set and status bytes of Part III §2.

2 Flagged Cross-Document Reconciliations

Two points of disagreement between this document and the Firmware Engineering Specification are recorded here explicitly, consistent with that document’s own practice of flagging rather than silently resolving cross-document discrepancies.

2.1 Resolved in Practice: Plaintext Capacity (Firmware OI-1)

The Firmware Engineering Specification records an open item (OI-1) noting two different figures for the secure plaintext ceiling: an originally intended 227 bytes (derived from the 255-byte maximum `Lc`), versus an as-built enforcement of 199 bytes (derived by applying the same 227-byte figure to `Lc` itself, i.e. to the complete encrypted package). **This application has already made a choice:** it enforces 199 bytes (§6.3 of Part I), matching the firmware’s as-built behavior rather than its originally stated intent. **Recommendation:** the firmware specification should adopt 199 bytes as the frozen figure and close OI-1 accordingly, since an implementation on the other end of the wire has now shipped against it; alternatively, if the firmware is changed to enforce $Lc \leq 255$ instead (restoring the original 227-byte intent), this application would need a corresponding change before it could use the additional 28 bytes of headroom. Until one of those happens, 199 bytes is the figure both ends of the wire actually agree on today.

2.2 Not Yet Resolved: C-APDU Framing (Le Byte)

The Firmware Engineering Specification’s Transport part states that a Short APDU C-APDU is “bounded at 261 bytes (CLA+INS+P1+P2+Lc+255 data bytes+Le)” – a formula that includes a trailing Le byte, consistent with ISO/IEC 7816-4 Case 4 framing. **This application never sends a trailing Le byte** (Part I §3 of this document): every C-APDU it transmits is Case 1 (no data) or Case 3 (data, no Le), bounding its own maximum C-APDU at 260 bytes, not 261.

This is a genuine, currently unresolved discrepancy between the two documents’ stated framing, not merely a difference in how the same thing is described: one document’s formula assumes a byte the other document’s implementation never sends. In practice this has not caused an observed interoperability failure, which suggests the firmware’s Transport implementation does not actually require a trailing Le to parse an incoming C-APDU (its own C-APDU field table, notably, does not list an Le field at all, despite the accompanying prose formula including one) – but this specification does not treat an unverified absence of failure as confirmation. **Recommendation:** the firmware Transport part’s C-APDU bound should be corrected to 260 bytes (or its field table amended to show why 261 is in fact correct), so the two documents state the same number for the same thing.

3 Security Properties

Property	Mechanism
Mutual device authentication	Ed25519 signatures over the full handshake transcript
Confidentiality	AES-256-GCM with directional keys
Integrity	AES-GCM authentication tag (16 B per message)
Key agreement	Ephemeral X25519, fresh per session
Key derivation	HKDF-SHA256, challenges bound into the salt
Replay protection (handshake)	Fresh challenges in the signed transcript
Forward secrecy	Fresh X25519 keypair per session, destroyed at termination
Reflection resistance	Distinct directional keys K_{p2e}/K_{e2p} (Part I §5.1)
Provisioning integrity	Deferred (never skipped) signature verification; persistence gated on that verification succeeding (Part III §1)

These properties are symmetric with, and depend on, the firmware’s own security properties (Firmware Engineering Specification, Part I §10); neither document’s properties hold in isolation without the other end of the wire honoring its half of the same handshake.

Part VII

Document Set History

1 Provenance and Scope of This Revision

This document is the first revision of the *Mobile Application Engineering Specification*. It is a new document, not a supersession of any prior specification, and covers the Phone-side counterpart to the *Firmware Engineering Specification*. Presentation (UI screens, widgets, and visual design) is deliberately excluded from this revision by request; a design-system and screen-architecture reference exists separately and is not incorporated here.

This document introduces two flagged cross-document reconciliations not previously recorded anywhere (§2): the de facto resolution of the firmware’s OI-1 plaintext-budget item from this application’s side, and a newly identified C-APDU framing discrepancy (the **Le** byte) that has no corresponding entry in the firmware document’s open-items list yet.

2 Open Items Tracked by This Revision

ID	Item	Status
OI-M1	C-APDU framing bound: 260 vs. 261 bytes (Le byte)	Flagged in §2; recommend correcting the firmware document; no observed interoperability failure to date
OI-M2	Firmware OI-1 (plaintext budget)	This application already enforces 199 bytes; recommend the firmware specification formally adopt the same figure and close its own open item

3 Revision History

Version	Change	Date
0.1	Initial revision. Specifies the wire protocol, session control and transport, provisioning and identity persistence, and application state architecture for the Phone-side mobile application, as the counterpart to the Firmware Engineering Specification. Presentation-layer content omitted by request. Two cross-document discrepancies identified and flagged (OI-M1, OI-M2) rather than silently resolved.	August 4, 2026